

Implementation Tasks — Phased & Assignable

This document breaks the Immediate High-Impact Fix and related follow-ups into discrete, grouped tasks that can be assigned to different developers and executed sequentially. Each task includes the goal, detailed steps, prerequisites, owner (placeholder), estimated effort, dependencies, verification / acceptance criteria, and rollback guidance.

Use this file as the single source of truth for sprint planning. Complete tasks in phase order and mark them done as you go.

Phase A — Foundations (Order: A1 → A2 → A3 → A4)

These tasks remove secrets from source control, make settings environment-driven, and add minimal CI gating.

A1 — Create `config/settings/` package and move settings

- Goal: Replace single `config/settings.py` with `config/settings/` containing `base.py`, `development.py`, `production.py`, `local.py` and an `\_\_init\_\_.py` selector.
- Owner: @backend-dev (placeholder)
- Effort: 2–4 hours
- Prerequisites: Read current `config/settings.py`. Make a small branch.
- Steps:
 1. Create `config/settings/` and move code into `base.py`.
 2. Create `development.py` and `production.py` that import `\*` from `base.py` then override values.
 3. Add `config/settings/\_\_init\_\_.py` which selects configuration via env var `DJANGO\_ENV`.
 4. Update `manage.py`, `wsgi.py`, `asgi.py` as necessary to default to `config.settings.development` in local dev when `DJANGO\_ENV` is unset.
 5. Run `python manage.py check`.
- Dependencies: None
- Acceptance criteria:
 - `python manage.py check` passes using development settings.
 - No secrets are hard-coded in `production.py`.
- Rollback: Revert branch; no DB changes required.

A2 — Add environment loader and `.env.example`

- Goal: Standardize env parsing and provide `.env.example` for onboarding.
- Owner: @backend-dev
- Effort: 1–2 hours
- Prerequisites: A1 completed
- Steps:
 1. Add `django-environ` to `requirements.txt` (or `python-dotenv`).
 2. At top of `base.py`, load `environ.Env()` and `environ.Env.read\_env()`.
 3. Define required env variables list (`REQUIRED\_ENV\_VARS`) and assert presence in production mode.
 4. Add `.env.example` at repo root with annotated keys (no values). See `docs/ARCHITECTURE.md` for suggested keys.
 5. Add short snippet to `README.md` showing how to `copy .env.example .env` and run server.
- Dependencies: A1
- Acceptance criteria:
 - Local dev can copy `.env.example` to `.env` and run `manage.py runserver`.
 - Production start-up fails if required env vars are missing (when `DEBUG=False`).
- Rollback: Revert changes to `base.py` and remove `.env.example` if needed.

A3 — Update `.gitignore` and remove `db.sqlite3` from VCS

- Goal: Prevent secrets and local DB from being committed.
- Owner: @devops
- Effort: 30–60 minutes

- Prerequisites: A1, A2
 - Steps:
 1. Add `.env`, `db.sqlite3`, build artifacts to `.gitignore`.
 2. Remove `db.sqlite3` from git history for new commits (but do NOT rewrite history yet).
 3. Commit `.gitignore` and push.
 - Dependencies: A2
 - Acceptance criteria:
 - `git status` shows no `.env` or `db.sqlite3` tracked.
 - Repository contains `.env.example` but not `.env`.
 - Rollback: Re-add `db.sqlite3` if needed (not recommended).
- > NOTE: If secrets were already committed historically, see A4.

A4 — CI: Add minimal secret-check + tests workflow

- Goal: Prevent secrets and unsafe settings from entering the main branch; run tests & lint on PRs.
- Owner: @devops / @backend-dev
- Effort: 2–6 hours
- Prerequisites: A1–A3
- Steps:
 1. Add `.github/workflows/ci.yml` (or `ci/secret-check.yml`) to run on PRs.
 2. Steps: checkout, setup python, pip install -r requirements.txt, run a simple grep-based secret check, run `pytest -q`, run `ruff check .`
 3. Optionally add `gitleaks-action` to the workflow for improved scanning.
- Dependencies: A1, A2, A3
- Acceptance criteria:
 - Workflow runs on PRs, fails when `SECRET_KEY` is found in tracked config files, and runs tests/lint.
 - Rollback: Disable workflow until adjusted.

Phase B — Operations & Reproducibility (Order: B1 → B2 → B3)

These tasks make the environment reproducible and prepare for scaling.

B1 — Pin dependencies and enable vulnerability scanning

- Goal: Produce a pinned `requirements.txt` and enable Dependabot/safety scanning.
- Owner: @backend-dev / @security
- Effort: 2–4 hours
- Prerequisites: A4
- Steps:
 1. Create `requirements.in` (top-level unpinned) and run `pip-compile` to produce `requirements.txt`, or pin versions manually.
 2. Add `dependabot.yml` to `.github` for dependency updates.
 3. Add `safety` or enable GitHub's Dependabot alerts.
- Dependencies: A4
- Acceptance criteria:
 - CI installs exact pinned versions and dependency alerts are enabled.
 - Rollback: Revert to previous requirements.

B2 — Containerize: Add `Dockerfile` and `docker-compose.yml`

- Goal: Provide reproducible local and CI environment with Postgres + Redis.
- Owner: @platform-engineer
- Effort: 1–2 days
- Prerequisites: A1–A4
- Steps:
 1. Add `Dockerfile` using Python slim and `gunicorn`.
 2. Add `docker-compose.yml` with services: `web`, `postgres`, `redis`.
 3. Document `docker-compose up --build` steps in README.
- Dependencies: B1
- Acceptance criteria:

- `docker-compose up` brings app and DB up; migrations run; app reachable at `localhost:8000`.
- Rollback: Provide fallback local venv instructions.

B3 — Background processing: Celery + Redis scaffolding

- Goal: Add Celery and Redis, convert one webhook to background task as example.
- Owner: @backend-dev / @integration-dev
- Effort: 2–5 days
- Prerequisites: B2
- Steps:
 1. Add Celery app scaffold in project (e.g., `config/celery.py`) and wire to Django.
 2. Configure Redis URL via env var and include in `docker-compose.yml`.
 3. Convert one payment webhook or signal handler to enqueue a Celery task.
 4. Add a healthcheck for Celery worker in docker-compose.
- Dependencies: B2
- Acceptance criteria:
 - Worker processes tasks from queue; webhook endpoint returns 200 quickly while work is processed asynchronously.
- Rollback: Revert webhook handling to synchronous if urgent.

Phase C — Observability & Reliability (Order: C1 → C2)

Focus on monitoring, error tracking, and strong testing.

C1 — Integrate Sentry and structured logging

- Goal: Add Sentry DSN support and structured JSON logging.
- Owner: @observability
- Effort: 1–2 days
- Prerequisites: A2, B2
- Steps:
 1. Add `sentry-sdk` to `requirements`.
 2. Configure `SENTRY\_DSN` env var and initialize Sentry in `base.py` or `wsgi.py`.
 3. Standardize logging format (JSON) and add a `/health` endpoint.
- Dependencies: A2, B2
- Acceptance criteria:
 - Errors and performance traces appear in Sentry for staging when `SENTRY\_DSN` is set.
- Rollback: Disable Sentry initialization.

C2 — Expand tests & add webhook contract tests

- Goal: Increase confidence with unit, integration, and contract tests for payments and webhooks.
- Owner: @qa / @backend-dev
- Effort: 1–3 weeks (incremental)
- Prerequisites: A4, B3
- Steps:
 1. Add parameterized unit tests covering payment service adapters.
 2. Add contract tests that assert webhook payloads are handled (e.g., using `requests-mock` or Pact-like approach).
 3. Add smoke-end-to-end test for signup and a payment flow (can be lightweight).
- Dependencies: B3
- Acceptance criteria:
 - Tests run in CI and coverage meets team target (e.g., 60–80% as a progressive goal).
- Rollback: Revert failing tests while fixing production code.

Phase D — Developer Experience & Polish (Order: D1 → D2)

Finish with tooling and documentation.

D1 — Add pre-commit hooks, linters, type checks

- Goal: Improve code quality with `pre-commit`, `ruff`, `black`, `isort`, and `mypy`.
- Owner: @backend-dev
- Effort: 1–3 days
- Prerequisites: A4, B1
- Steps:
 1. Add `.pre-commit-config.yaml` with `ruff`, `black`, `isort` hooks.
 2. Configure `mypy` and add `mypy.ini` for gradual typing.
 3. Document how to run locally and enforce in CI.
- Acceptance criteria:
 - Pre-commit prevents style regressions locally and CI runs linters.
- Rollback: Disable pre-commit temporarily if needed.

D2 — Final docs & handoff

- Goal: Provide final onboarding docs, `CONTRIBUTING.md`, `SECURITY.md`, and handoff notes.
- Owner: @tech-writer / @team-lead
- Effort: 1–2 days
- Prerequisites: completion of Phases A–C
- Steps:
 1. Update `README.md` with full dev setup: virtualenv, `.env` creation, run commands, Docker workflow.
 2. Add `CONTRIBUTING.md` with PR and code-review process.
 3. Add `SECURITY.md` describing secret handling and incident response.
- Acceptance criteria:
 - New developer can follow README and run app locally and in Docker.
 - Team has documented process for secret handling and rotation.

Assignment & Execution Notes

- Execute tasks in the phase order listed. Within each phase tasks are ordered; do not start a dependent task until its dependencies are complete.
- Use feature branches for each task (e.g., `feature/a1-settings-package`) and open PRs with a checklist referencing the acceptance criteria above.
- Each PR must pass the CI workflow (A4) before merging.
- For destructive operations (rewriting history), schedule a maintenance window and coordinate with all developers.

How to mark progress

- Update the repository's issue tracker or the team's project board using the task names above.
- Use this document as the source-of-truth; update owners and estimated times as you refine.

(End of Implementation Tasks)